
SOFTWARE DEVELOPER TRAINING

Curso 0: Git, GitHub y Herramientas del Desarrollador

Tipo: Prerequisito

Modulos: 5

Objetivo general

Preparar el entorno de trabajo del estudiante e introducirlo en el uso de Git y GitHub como herramientas fundamentales que utilizará a lo largo de todo el programa de formación. Al finalizar este curso, el estudiante será capaz de usar la terminal, tener su entorno configurado, y manejar el flujo completo de Git: clonar, crear ramas, hacer commits, push y abrir Pull Requests hacia develop.

Nota importante

El contenido presentado en este curso es introductorio y orientativo. Su propósito es proporcionar una base conceptual y práctica sobre cada tema, pero no pretende ser exhaustivo. Se espera que el estudiante complemente este material con su propia investigación, consulte fuentes adicionales (documentación oficial, artículos, tutoriales) y profundice en los temas según su ritmo y curiosidad. La capacidad de investigar y aprender de forma autónoma es una habilidad fundamental en el desarrollo de software.

Módulo 1: La terminal / línea de comandos

Contenido

- **¿Qué es una terminal?** Definición, para qué sirve, por qué es imprescindible para un desarrollador.
- **Tipos de terminal:** CMD (Windows), PowerShell, Bash (Linux/Mac), terminal integrada en VS Code. Diferencias y cuándo usar cada una.
- **Navegación de archivos y carpetas:** comandos esenciales — `cd`, `ls` / `dir`, `pwd`, `mkdir`, `rmdir`, `rm`, `cp`, `mv`, `cat`, `clear`.
- **Rutas absolutas vs relativas:** cómo funcionan, ejemplos prácticos, errores comunes.
- **Variables de entorno:** qué son, para qué sirven, cómo ver las existentes (`echo $PATH`, `echo %PATH%`). Concepto básico sin profundizar en configuración avanzada.
- **Atajos útiles:** Tab (autocompletado), flecha arriba (historial), Ctrl+C (cancelar).

Actividad del modulo

Crear la siguiente estructura de carpetas usando **únicamente la terminal** (sin usar el explorador de archivos):

```
mi-practica/
├── documentos/
│   ├── notas.txt
│   └── resumen.txt
├── proyectos/
│   └── proyecto-01/
│       └── README.txt
└── recursos/
```

Navegar entre las carpetas, listar su contenido y capturar evidencia (screenshot de la terminal). Además, elaborar un **mapa conceptual** que clasifique los comandos aprendidos por categoría (navegación, archivos, carpetas, utilidades).

Módulo 2: Instalación y configuración del entorno de desarrollo

Contenido

- **Creación de cuenta de GitHub y acceso al repositorio del training:** antes de instalar cualquier herramienta, el estudiante debe asegurarse de tener una cuenta en GitHub, ya que será indispensable para todo el programa de formación. Si no tiene una, debe crearla en github.com utilizando una dirección de correo electrónico de **Gmail** o **Outlook**. Una vez creada la cuenta, debe proporcionar al instructor el correo electrónico asociado para que le envíe la invitación de colaborador al repositorio del training, el cual es privado. El estudiante recibirá la invitación por correo electrónico y deberá aceptarla. Este paso es requisito previo para las actividades prácticas de los módulos siguientes.
- **Git:** descarga e instalación desde git-scm.com. Verificación con `git --version`. Configuración inicial obligatoria (usar el mismo correo electrónico registrado en GitHub):

```
git config --global user.name "Tu Nombre"
git config --global user.email "tu@email.com"
```

- **Node.js:** descarga e instalación (versión LTS). Verificación con `node -v` y `npm -v`. Para qué lo usaremos más adelante.
- **.NET SDK 10:** descarga e instalación. Verificación con `dotnet --info`. Para qué lo usaremos más adelante.
- **Visual Studio 2026:** instalación con los workloads necesarios (ASP.NET, .NET desktop). Recorrido básico de la interfaz.
- **Visual Studio Code:** instalación y extensiones recomendadas (C#, ESLint, Prettier, GitLens, Material Icon Theme, Spanish Language Pack). Recorrido básico de la interfaz.
- **Cómo verificar todas las instalaciones desde terminal:** checklist de comandos.
- **Concepto de IDE vs editor de código:** diferencias fundamentales.

Actividad del modulo

1. Crear la cuenta de GitHub (si no la tiene) y enviar el correo asociado al instructor para recibir la invitación al repositorio del training. Aceptar la invitación una vez recibida.
2. Instalar todas las herramientas mencionadas.
3. Ejecutar todos los comandos de verificación y capturar evidencia (screenshot).
4. Elaborar un **cuadro comparativo** de Visual Studio 2026 vs Visual Studio Code: qué es cada uno, en qué se diferencian, ventajas, desventajas, y cuándo conviene usar cada uno.

Módulo 3: Git fundamentos

Contenido

- **¿Qué es un sistema de control de versiones?** Por qué existe, qué problema resuelve, analogía con "guardar versiones de un documento".
- **Git vs GitHub:** diferencia entre la herramienta local (Git) y la plataforma en la nube (GitHub).
- **Conceptos fundamentales:**
 - **Repositorio (repo):** carpeta rastreada por Git.
 - **Working directory:** donde editas archivos.
 - **Staging area (index):** donde preparas lo que va en el próximo commit.
 - **Commit:** una "foto" del estado del proyecto en un momento dado.
 - **Remote (origin):** el repositorio en GitHub.
- **Flujo básico de trabajo:**
 - a. `git clone <url>` — clonar un repositorio remoto. Para el repositorio del training, el estudiante ya debe tener acceso (configurado en el Módulo 2). El instructor proporcionará la URL.
 - b. Hacer cambios en archivos.
 - c. `git status` — ver el estado actual.
 - d. `git add <archivo>` o `git add .` — enviar al staging.
 - e. `git commit -m "mensaje"` — crear el commit.
 - f. `git push` — subir al remoto.
 - g. `git pull` — traer cambios del remoto.
- **El archivo `.gitignore`:** qué es, para qué sirve, ejemplos comunes (node_modules, bin, obj, .env).
- **Ver el historial:** `git log`, `git log --oneline`, `git log --graph`.
- **Deshacer cambios básicos:** `git checkout -- <archivo>`, `git reset HEAD <archivo>`.

Actividad del modulo

1. Elaborar un **diagrama de flujo** que represente el ciclo de vida de un archivo en Git (untracked → staged → committed → pushed), indicando qué comando se usa en cada transición.
2. **Ejercicio práctico:** Clonar el repositorio del training (ya se debe tener acceso desde el Módulo 2; consultar al instructor si aún no lo tiene), crear un archivo `curso-0/modulo-3/mi-primer-archivo.txt` con un texto libre, hacer add, commit con un mensaje descriptivo, y push.

Módulo 4: Ramas y Pull Requests

Contenido

- **¿Qué es una rama (branch)?** Analogía: "líneas paralelas de trabajo". Por qué son esenciales en el desarrollo de software.
- **La rama `main` y la rama `develop`:** convención de ramas principales. Por qué no se trabaja directamente en ellas.
- **Comandos de ramas:**
 - `git branch` — listar ramas.
 - `git branch <nombre>` — crear rama.
 - `git checkout <nombre>` o `git switch <nombre>` — cambiar de rama.
 - `git checkout -b <nombre>` o `git switch -c <nombre>` — crear y cambiar en un solo paso.
 - `git branch -d <nombre>` — eliminar rama local.
- **Convención de nombres de ramas para el training:**

curso-X/modulo-Y/descripcion-breve

Ejemplo: `curso-0/modulo-4/actividad-ramas`

- **Merge básico:** qué es, cómo funciona `git merge`, qué es un conflicto y cómo resolverlo de forma simple.
- **Push de ramas:** `git push -u origin <nombre-rama>`.
- **¿Qué es un Pull Request (PR)?** Definición, para qué sirve, por qué es el mecanismo estándar en equipos de desarrollo.
- **Flujo completo:**
 - a. Crear rama desde develop.
 - b. Trabajar y hacer commits.
 - c. Push de la rama.
 - d. Abrir PR en GitHub hacia develop.
 - e. Revisión y merge.

Actividad del modulo

1. Elaborar un **mapa conceptual** del flujo de trabajo con ramas: desde la creación de la rama hasta el merge del PR, incluyendo los comandos involucrados en cada paso.

2. **Ejercicio práctico:** Crear la rama `curso-0/modulo-4/practica-ramas`, agregar un archivo `reflexion.txt` explicando con sus propias palabras por qué las ramas son útiles, hacer commit, push, y abrir un PR hacia `develop`.

Módulo 5: GitHub y Markdown

Contenido

- **La plataforma GitHub:**
 - Interfaz web: repos, código, commits, branches, PRs, issues.
 - Perfil del desarrollador: por qué importa, qué muestra.
 - Explorar repositorios de otros: stars, forks, contribuciones.
- **Sintaxis Markdown:**
 - Encabezados (`#`, `##`, `###`).
 - Texto en **negrita**, *cursiva*, `~~tachado~~`.
 - Listas ordenadas y desordenadas.
 - Links e imágenes.
 - Bloques de código (inline y multilinea con syntax highlighting).
 - Tablas.
 - Citas (`>`).
 - Separadores (`---`).
- **Buenos mensajes de commit:**
 - Convención: verbo en imperativo, breve, descriptivo.
 - Ejemplos buenos vs malos.
 - Formato: `tipo: descripción` (feat, fix, docs, chore, etc.).
- **El archivo README.md:** qué es, por qué todo repositorio debe tener uno, qué debe contener.

Actividad del modulo

1. Investigar y elaborar una **matriz** con la sintaxis Markdown: elemento, sintaxis, ejemplo visual y cuándo usarlo.
2. **Ejercicio práctico:** En la rama `curso-0/modulo-5/markdown`, crear un archivo `markdown-practica.md` que demuestre al menos 8 elementos diferentes de Markdown (encabezados, listas, tabla, código, imagen, link, negrita/cursiva, citas). Commit, push y PR.

Actividad final del Curso 0

"Mi carta de presentación en el repo"

Desde la rama `curso-0/actividad-final`:

1. Crear dentro del repositorio la carpeta con su nombre (ejemplo: `estudiantes/wilfran/`).
2. Dentro de esa carpeta, crear un archivo `README.md` que contenga:
 - Nombre completo.
 - Una breve presentación personal.
 - Por qué quiere aprender a programar.
 - Qué espera de este training.
 - Qué herramientas instaló (con versiones).
 - Un listado de los comandos Git que aprendió, organizados por categoría.
3. El archivo debe utilizar correctamente al menos: encabezados, listas, negrita, una tabla y un bloque de código.
4. Hacer commits con mensajes que sigan la convención aprendida.
5. Push y abrir PR hacia `develop`